Algorithmique. Rappels.

Nous allons revoir quelques définitions importantes de première en s'appuyant sur les algorithmes classiques de recherche et de tris.

I) ALGORITHME NAÏF DE RECHERCHE : LA RECHERCHE EN TABLE :

Pour rechercher un élément dans une table on pourrait simplement parcourir tout simplement le tableau jusqu'à rencontrer la valeur recherchée. C'est ce que l'on appelle la recherche en table.

```
def recherche(liste, élément):
    """Recherche d'un élément dans une liste

    Arguments
    ---------
    liste: liste d'entiers
    élément: entier

    Returns
    -------
    int: l'indice de l'élément si trouvé ou -1 sinon

    """
    i = 0
    while i < len(liste) and liste[i] != élément:
        i += 1
    if i < len(liste):
        return i
    else:
        return -1

# Quelques tests
assert recherche([1], 1) == 0
assert recherche([1,6,5], 5) == 2
assert recherche([1,6,5], 7) == -1
```

Même si cet algorithme simple semble faire le travail, il n'est en pratique pas du tout utilisé car il n'est pas du tout efficace.
Pour mesurer l'efficacité d'un algorithme, on utilise la notion de **complexité**.

II) NOTION DE COMPLEXITE :

Si je donne à mon programme une entrée de taille N, quel est l'ordre de grandeur, en fonction de N, du nombre d'opérations qu'il va effectuer ?

1°) Définition :

La complexité d'un algorithme est le nombre d'opérations élémentaires *(opérations arithmétique, comparaisons, affectations...)* effectuées pour obtenir un résultat.

Si on prend l'exemple de l'algorithme précédent, on se rend compte que cela dépend des cas. Expliquez...

Pour pouvoir faire des comparaisons entre algorithmes, l'informaticien étudie souvent la **complexité dans le pire des cas**.

2°) Complexité de la recherche linéaire :

Regardons ce que cela donne dans le cas de notre recherche en table, le pire des cas correspond au cas où l'élément n'est pas dans le tableau.
Etudions chacune des étapes pour compter les opérations élémentaires effectuées.

```
i = 0                                  # 1 opération
while i < len(liste) and liste[i] != élément:   # 2 opérations × N
   i += 1                              # 1 opération  × N
if i < len(liste):                     # 1 opération
   return i
else:
   return -1                           # 1 opération
```

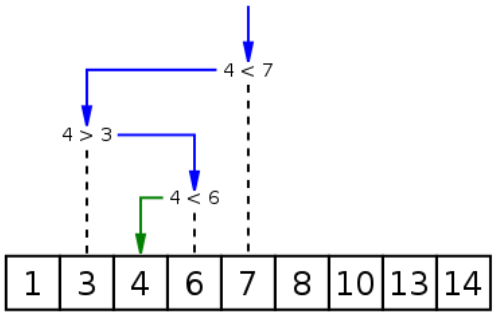
On obtient donc : 3N+3 opérations.

Dans la pratique, on ne s'intéresse pas au détail de la complexité, mais on utilise une **notation asymptotique** dite *"grand O"*. Cette notation permet de comparer très simplement les algorithmes.
Les facteurs multiplicatifs et additifs sont négligés, on dit que notre algorithme a une complexité grand O de N notée : O(N).

On parle d'algorithme linéaire : son temps d'exécution croit linéairement avec la taille de l'entrée.

III) UN ALGORITHME EFFICACE : LA RECHERCHE DICHOTOMIQUE :

Quand on cherche un mot dans le dictionnaire, on ne va pas le chercher en les lisant un par un, on va utiliser la méthode de recherche dichotomique vue en première.
Cette méthode est possible dans le cas où les données ont été préalablement triées, ce pour quoi il existe également des algorithmes efficaces.
Cette image illustre la recherche de l'élément 4 dans tableau trié.



1°) Implémentation en Python :

Voici un exemple d'implémentation en Python :

```
def recherche_dichotomique(liste, élément):
   # on initialise les indices début et fin aux extrémités de la liste
   début = 0
   fin = len(liste)

   while début <= fin:
      # On se place au milieu de la liste
      milieu = (début + fin) // 2 # il, s'agit d'une division entière

      if liste[milieu] == élément:
         print(élément, "trouvé à l'indice:", milieu , liste[milieu])
         return True
         # on arrête la boucle
         début = fin - 1
      elif liste[milieu] < élément:
         début = milieu + 1
      else:
         fin = milieu - 1
```

```
    print(élément, "non trouvé")
    return False
```

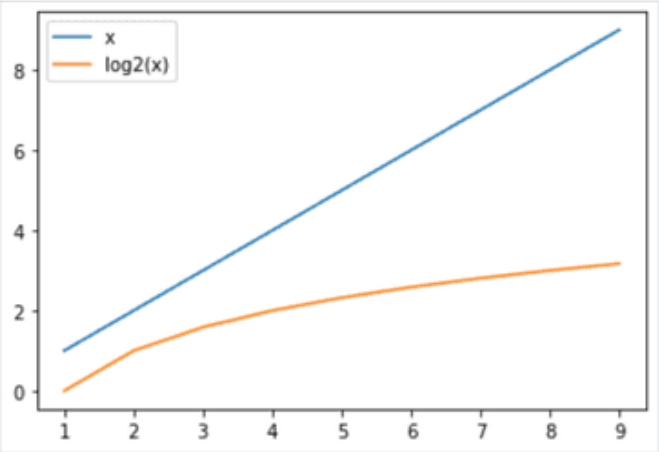
Cet algorithme est beaucoup plus efficace, sa complexité (asymptotique dans le pire des cas) est : $O(\log_2(N))$, avec N qui représente la longueur de la liste.

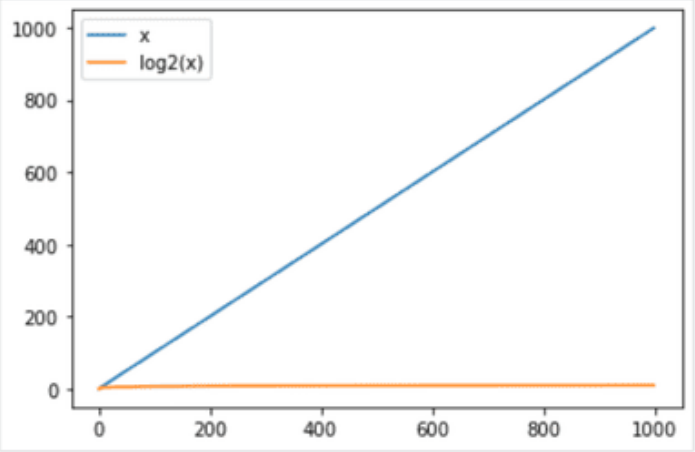
2°) Complexité de la recherche dichotomique :

Calculer comme précédemment le nombre d'opérations élémentaires effectuées par cet algorithme dans le pire des cas, et retrouver la complexité donnée.

```
début = 0
fin = len(liste)

while début <= fin:
    # On se place au milieu de la liste
    milieu = (début + fin) // 2

    if liste[milieu] == élément:

        return True
        # on arrête la boucle
        début = fin - 1
    elif liste[milieu] < élément:
        début = milieu + 1
    else:
        fin = milieu - 1

return False
```

Ceci fait une énorme différence notamment lorsque la taille des données augmente :





IV) CORRECTION D'UN ALGORITHME :

Pour rappel, un algorithme est une suite d'instructions permettant d'obtenir un résultat.

La **correction d'un algorithme** est une démonstration qui prouve que l'algorithme permet bien d'obtenir le résultat souhaité.

Nous allons utiliser une méthode répandue semblable au *raisonnement par récurrence* fondée sur la recherche d'un **invariant de boucle**.

Pour prouver la correction nous devons montrer les trois points suivants :

1. **Initialisation :** l'invariant est vrai avant la première itération.
2. **Conservation :** si l'invariant est vrai avant une itération, il restera vrai après l'itération.
3. **Terminaison :** la boucle se termine et nous donne le résultat attendu.

Nous allons appliquer cette méthode aux algorithmes de tris vus en première.

1°) Correction du tri par sélection :

On rappelle le principe de l'algorithme.

Sur un tableau de n éléments (numérotés de 0 à n), le principe du tri par sélection est le suivant :

- rechercher le plus petit élément du tableau, et l'échanger avec l'élément d'indice 0 ;
- rechercher le second plus petit élément du tableau, et l'échanger avec l'élément d'indice 1 ;
- continuer de cette façon jusqu'à ce que le tableau soit entièrement trié.

En voici une implémentation en python.

```
def tri_selection(t):
    n = len(t)
    for i in range(n-1):
        i_min = i
        for j in range(i+1, n):
            if t[j] < t[i_min]:
                i_min = j
        if i_min != i:
            # échanger t[i] et t[min]
            t[i], t[i_min] = t[i_min], t[i]
    return t
```

Voici les états successifs du tableau après chaque tour de boucle avec en entrée [11, 25, 12, 22, 64] :

```
[11, 25, 12, 22, 64]
[11, 12, 25, 22, 64]
[11, 12, 22, 25, 64]
[11, 12, 22, 25, 64]
```

On voit qu'avec un tableau à 5 valeurs, 4 tours de boucles (externe) suffisent à trier le tableau.

Démonstration de la correction

L'invariant de boucle consiste à montrer que si les i premiers éléments du tableau sont triés avant l'itération, alors les i+1 premiers éléments seront triés après une itération.

1. **Initialisation :** Au départ, i=0, le sous-tableau trié de gauche ne contient aucun élément []. Il est donc forcément trié.
2. **Conservation :** Lorsqu'on considère le tour de boucle i, le tableau est déjà trié pour les indices 0 à i−1. Grâce à la boucle interne, on trouve le plus petit élément parmi les éléments d'indice i à n (tous plus grands que l'élément d'indice i−1), et on le place à l'indice i. Après le tour de boucle, le tableau sera donc trié pour les indices de 0 à i.
3. **Terminaison :** la boucle se termine lorsqu'on arrive à l'avant dernier élément du tableau. Le tableau est trié pour les éléments d'indice 0 à n−2, et le dernier élément d'indice n−1 est forcément plus grand que l'élément d'indice n–2. Le tableau est donc entièrement trié.

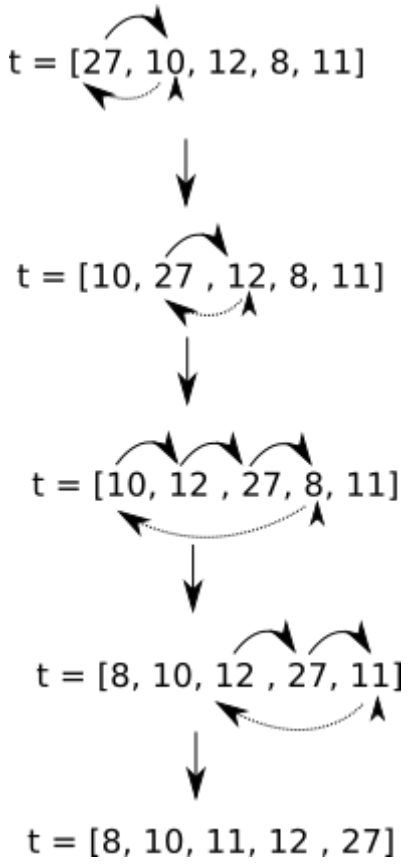
2°) Étude du tri par insertion :

La partie exercices propose une étude complète de l'algorithme de tri par insertion vu également en première.

Dans l'algorithme, on parcourt le tableau à trier du début à la fin. Au moment où on considère le i-ème élément, les éléments qui le précèdent sont déjà triés. Pour faire l'analogie avec l'exemple du jeu de cartes, lorsqu'on est à la i-ème étape du parcours, le i-ème élément est la carte saisie, les éléments précédents sont la main triée et les éléments suivants correspondent aux cartes encore mélangées sur la table.

L'objectif d'une étape est d'insérer le i-ème élément à sa place parmi ceux qui précèdent. Il faut pour cela trouver où l'élément doit être inséré en le comparant aux autres, puis décaler les éléments afin de pouvoir effectuer l'insertion. En pratique, ces deux actions sont fréquemment effectuées en une passe, qui consiste à faire « remonter » l'élément au fur et à mesure jusqu'à rencontrer un élément plus petit.

On peut résumer le principe de fonctionnement de l'algorithme de tri par insertion avec le schéma suivant :




```
def tri_insertion(t: list):
    n = len(t)
    for i in range(1, n):
        x = t[i]
        j = i
        while j > 0 and t[j-1] > x:
            t[j] = t[j-1]
            j = j - 1
        t[j] = x
    return t
```

V) <u>EXERCICES</u> :

<u>Exercice 1</u> :
Maitriser les boucles en Python.
Si besoin consulter la page de cours de première sur les boucles.
Prévoir la sortie des instructions suivantes :

```
for i in range(5):
    print(i)
```

```
for i in range(1, 5):
    print(i)
```

```
for i in range(5, 1, -1):
    print(i)
```

```
for i in range(5):
    for j in range(i):
        print(j)
```

```
for i in range(5):
    j = i
```

```
    while j > 0:
        print(j)
        j -= 1
```

Écrire le code permettant d'obtenir les sorties suivantes :

```
O
OO
OOO
OOOO
OOOOO
OOOOOO
OOOOOOO
OOOOOOOO
OOOOOOOOO
```

```
0000000000
111111111
22222222
3333333
444444
55555
6666
777
88
9
```

```
  | |
-------
  | |
-------
  | |
```

<u>Exercice 2</u> :
Vérifier qu'une liste est triée.
Écrire le code d'une fonction appelée is_sorted qui prend en paramètres une liste et qui renvoie True si la liste est triée et False sinon.

1. Écrire en python le code de la fonction en la *prototypant*.
2. Proposer quelques tests d'assertion en pensant aux cas limites : liste vide, liste triée, éléments égaux...
3. Évaluer la complexité de l'algorithme dans le pire des cas.

<u>Exercice 3</u> :
Étude du tri par insertion.
On donne ci-dessous le code Python du tri par insertion :

```
def tri_insertion(t):
    n = len(t)
    for i in range(1, n):
        # ICI
        x = t[i]
        j = i
        while j > 0 and t[j-1] > x:
            t[j] = t[j-1]
            j = j - 1
        t[j] = x
        # LA
    return t
```

a) <u>Compréhension de l'algorithme</u> :

1. Écrire le prototype de cette fonction.

On va étudier l'appel de la fonction avec comme argument [11, 25, 12, 22, 64].

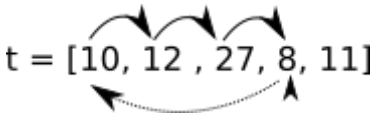
2. Écrire l'instruction permettant d'exécuter l'appel de la fonction avec la liste précédente.

3. Que représente n ? Quelle est sa valeur ?

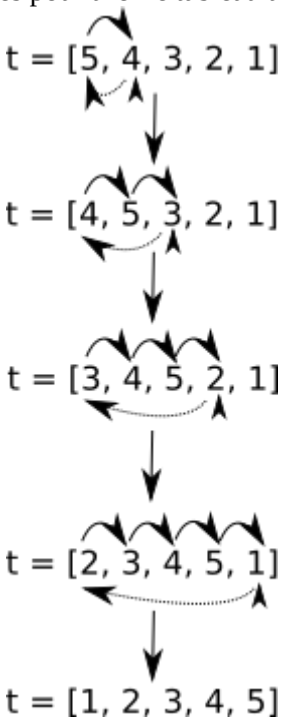
4. Quelles seront les valeurs prises par i données par l'instruction for i in range (1, n) ?

5. Algorithme et notion de décalage : prenons comme exemple, la liste : t = [5, 4, 3, 2, 1] (soit la pire des listes à trier, puisqu'elle est déjà triée dans l'ordre décroissant).

   Par souci de simplicité, nous allons directement nous intéresser au "nombre de décalages effectués" pour trier entièrement un tableau. On appelle "décalage" ce qui est symbolisé par une flèche noire sur le schéma ci-dessous :

   Pour l'étape ci-dessus nous avons 3 décalages (décalages du 10, du 12 et du 27). Nous ne tiendrons pas compte du "placement" du nombre en cours de traitement (8 dans notre exemple) symbolisé par la flèche en pointillé.
   Évaluons le nombre de décalages nécessaires pour trier le tableau t = [5, 4, 3, 2, 1].

   Combien a-t-on de décalages ? Faire le lien avec la longueur de la liste.

6. Dans un tableau, donner les états successifs du tableau aux points ICI et LA pour tous les tours de la boucle externe(for). Au pire, combien peut-on avoir de décalages ?

7. Expliquer le rôle de la boucle interne (while).

b) <u>Étude de la complexité</u> :

1. Rappeler la définition de la complexité.

2. Montrer que pour tout entier n, la somme des entiers de 1 à n vaut :
   $S = 1 + 2 + 3 + \cdots + (n-3) + (n-2) + (n-1) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$.
   (Voir cette animation si nécessaire.). Faire le lien avec le nombre de décalages nécessaires au tri.

3. En déduire la complexité du tri par insertion proposé.

4. En déduire qu'il s'agit d'un algorithme de complexité quadratique : $O(n^2)$.

c) Correction de l'algorithme :

Etablir la correction de l'algorithme. On rappelle que pour prouver la correction nous devons montrer les trois points suivants :

1. **Initialisation :** L'invariant est vrai avant la première itération.
2. **Conservation :** si l'invariant est vrai avant une itération, il restera vrai après l'itération.
3. **Terminaison :** la boucle se termine et nous donne le résultat attendu.